

FinShield AI: An End-to-End Machine Learning System for Real-Time Financial Fraud Detection in a Global Fintech Environment

In []:

Project Introduction

FinShield AI is an end-to-end machine learning project built for Zenvra International, an imaginary global fintech institution. It simulates a complete real-world fraud detection pipeline, from synthetic data generation through to a live, publicly accessible prediction web application, covering transactions across 40 countries, 7 regions, and 6 payment channels.

In []:

Project Aims & Objectives

The primary aim is to build and deploy a machine learning system capable of detecting fraudulent financial transactions in real time. This was achieved by generating a realistic synthetic dataset, cleaning and transforming it, conducting exploratory analysis, engineering predictive features, training and comparing three ML classifiers, and deploying the best-performing model as an interactive web application accessible to anyone globally.

Installing Dependencies

```
In [11]: import subprocess
subprocess.run([
    "pip", "install", "pandas", "numpy", "matplotlib", "seaborn",
    "scikit-learn", "xgboost", "imbalanced-learn", "shap", "joblib", "streamlit"
], check=True)
print(" All dependencies installed.")
```

All dependencies installed.

In []:

Create Project Folder Structure

```
In [12]: import os

# Project root on my Desktop System
username = os.getlogin()
PROJECT_ROOT = rf"C:\Users\{username}\Desktop\Data Analytics Projects\finshield_ai"

folders = [
    r"data\raw",
    r"data\processed",
    r"notebooks",
    r"models",
    r"app",
    r"outputs\figures",
    r"outputs\reports",
]

for f in folders:
    full_path = os.path.join(PROJECT_ROOT, f)
    os.makedirs(full_path, exist_ok=True)
    print(f"    Created: {full_path}")

print(f"\n Project structure ready at:\n    {PROJECT_ROOT}")
```

```
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\data\raw
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\data\processed
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\notebooks
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\models
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\app
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\outputs\figures
Created: C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\outputs\reports
```

Project structure ready at:
C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai

In []:

Global Imports & PROJECT_ROOT

```
In [1]: import os
import json
import joblib
import warnings
from datetime import datetime, timedelta

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    classification_report, confusion_matrix,
    roc_auc_score, roc_curve,
    average_precision_score, f1_score
)
from imblearn.over_sampling import SMOTE
from xgboost import XGBClassifier
import shap

warnings.filterwarnings('ignore')
sns.set_theme(style='white', palette='muted')
plt.rcParams['figure.dpi'] = 120

# Define PROJECT_ROOT – used by every cell below
username = os.getlogin()
PROJECT_ROOT = rf"C:\Users\{username}\Desktop\Data Analytics Projects\finshield_ai"

# Helper: build any path inside the project cleanly
def p(*args):
    """Shorthand for os.path.join(PROJECT_ROOT, *args)"""
    return os.path.join(PROJECT_ROOT, *args)

print(" All libraries imported.")
print(f" PROJECT_ROOT → {PROJECT_ROOT}")
```

All libraries imported.
PROJECT_ROOT → C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai

In []:

SYNTHETIC DATA GENERATION

Reference Data

```
In [27]: np.random.seed(42)

N = 3000 # total transactions

# Countries mapped to their global region
countries = {
    'United States': 'North America', 'Canada': 'North America',
    'Mexico': 'North America', 'Brazil': 'South America',
    'Argentina': 'South America', 'Colombia': 'South America',
    'Peru': 'South America', 'United Kingdom': 'Europe',
    'Germany': 'Europe', 'France': 'Europe', 'Netherlands': 'Europe',
    'Spain': 'Europe', 'Italy': 'Europe', 'Sweden': 'Europe',
    'Poland': 'Europe', 'Ukraine': 'Europe', 'Russia': 'Europe',
    'Nigeria': 'Africa', 'South Africa': 'Africa', 'Kenya': 'Africa',
    'Ghana': 'Africa', 'Egypt': 'Africa', 'Morocco': 'Africa',
    'Tanzania': 'Africa', 'India': 'Asia', 'China': 'Asia',
    'Japan': 'Asia', 'Singapore': 'Asia', 'Indonesia': 'Asia',
    'Philippines': 'Asia', 'Malaysia': 'Asia', 'Thailand': 'Asia',
    'Vietnam': 'Asia', 'Pakistan': 'Asia', 'Bangladesh': 'Asia',
    'Australia': 'Oceania', 'New Zealand': 'Oceania',
    'United Arab Emirates': 'Middle East', 'Saudi Arabia': 'Middle East',
    'Turkey': 'Middle East',
}

country_names = list(countries.keys())
channels = ['mobile_app', 'web', 'POS', 'ATM', 'USSD', 'bank_transfer']
merchant_cats = [
    'e-commerce', 'grocery', 'utilities', 'travel', 'entertainment',
    'restaurants', 'healthcare', 'financial_services', 'crypto_exchange',
    'gaming', 'remittance', 'fuel', 'education', 'subscription'
]
devices = ['Android', 'iOS', 'Windows', 'macOS', 'Linux', 'Unknown']
```

```

payment_methods = ['debit_card', 'credit_card', 'wallet', 'bank_transfer', 'crypto', 'BNPL']
kyc_tiers        = ['unverified', 'basic', 'enhanced', 'premium']

```

```

print(f" Reference data ready.")
print(f"    {len(country_names)} countries across {len(set(countries.values()))} regions.")

```

```

Reference data ready.
40 countries across 7 regions.

```

In []:

Generate Transactions

```

In [28]: def generate_transactions(n):
rows = []
start_date = datetime(2023, 1, 1)

hour_weights = np.array([
    0.01, 0.01, 0.01, 0.01, 0.01, 0.01,
    0.03, 0.04, 0.05, 0.06, 0.07, 0.07,
    0.07, 0.07, 0.06, 0.06, 0.06, 0.06,
    0.05, 0.05, 0.04, 0.03, 0.02, 0.01
])
hour_weights = hour_weights / hour_weights.sum()

country_weights = np.array([
    0.18, 0.06, 0.02, 0.05, 0.02, 0.02, 0.01,
    0.07, 0.05, 0.04, 0.03, 0.03, 0.02, 0.02, 0.01, 0.01,
    0.04, 0.03, 0.02, 0.02, 0.02, 0.01, 0.01,
    0.05, 0.04, 0.02, 0.02, 0.02, 0.02, 0.02, 0.01, 0.01, 0.01, 0.01,
    0.02, 0.01,
    0.02, 0.01, 0.01
])
country_weights = country_weights / country_weights.sum()

channel_weights = np.array([0.35, 0.28, 0.15, 0.10, 0.07, 0.05])
channel_weights = channel_weights / channel_weights.sum()
device_weights = np.array([0.35, 0.30, 0.15, 0.10, 0.05, 0.05])
device_weights = device_weights / device_weights.sum()
payment_weights = np.array([0.30, 0.28, 0.18, 0.12, 0.07, 0.05])
payment_weights = payment_weights / payment_weights.sum()
kyc_weights = np.array([0.10, 0.35, 0.40, 0.15])
kyc_weights = kyc_weights / kyc_weights.sum()
distinct_weights = np.array([0.65, 0.20, 0.08, 0.04, 0.03])
distinct_weights = distinct_weights / distinct_weights.sum()
device_flag_w = np.array([0.80, 0.20])
device_flag_w = device_flag_w / device_flag_w.sum()
ip_weights = np.array([0.85, 0.15])
ip_weights = ip_weights / ip_weights.sum()

amount_means = {
    'e-commerce': 5.2, 'travel': 6.5, 'grocery': 4.0,
    'crypto_exchange': 7.5, 'remittance': 5.8, 'gaming': 3.5,
    'utilities': 4.5, 'healthcare': 5.5, 'financial_services': 6.8,
    'restaurants': 3.8, 'entertainment': 4.2, 'fuel': 4.0,
    'education': 5.0, 'subscription': 3.0
}

for i in range(n):
    days_offset = np.random.randint(0, 540)
    hour = int(np.random.choice(range(24), p=hour_weights))
    ts = start_date + timedelta(
        days=int(days_offset),
        hours=hour,
        minutes=np.random.randint(0, 60)
    )

    country = np.random.choice(country_names, p=country_weights)
    region = countries[country]
    channel = np.random.choice(channels, p=channel_weights)
    merchant = np.random.choice(merchant_cats)
    device = np.random.choice(devices, p=device_weights)
    payment = np.random.choice(payment_methods, p=payment_weights)
    kyc = np.random.choice(kyc_tiers, p=kyc_weights)

    amount = round(np.random.lognormal(
        mean=amount_means.get(merchant, 4.5), sigma=0.9), 2)
    amount = min(amount, 250000)

    account_age_months = np.random.randint(1, 120)
    tx_count_30d = np.random.poisson(lam=12)
    avg_tx_amount_30d = round(np.random.lognormal(4.8, 0.7), 2)
    failed_tx_7d = np.random.poisson(0.5)

```

```

login_attempts_24h = np.random.poisson(1.2)
distinct_countries = np.random.choice([1,2,3,4,5], p=distinct_weights)
is_new_device = np.random.choice([0, 1], p=device_flag_w)
is_night_tx = 1 if hour < 6 else 0
is_weekend = 1 if ts.weekday() >= 5 else 0
amount_vs_avg_ratio = round(amount / (avg_tx_amount_30d + 1e-6), 4)
velocity_flag = 1 if tx_count_30d > 40 else 0
ip_country_mismatch = np.random.choice([0, 1], p=ip_weights)

```

```

# — Strengthened fraud scoring —————
# Increments are larger so fraud cases cluster more distinctly
# around high-risk feature combinations, giving models clearer signal
fraud_prob = 0.05 # raised baseline from 0.03 to 0.05

```

```

if ip_country_mismatch: fraud_prob += 0.15 # was 0.08
if kyc == 'unverified': fraud_prob += 0.20 # was 0.10
if kyc == 'basic': fraud_prob += 0.05 # new
if is_new_device: fraud_prob += 0.12 # was 0.06
if is_night_tx: fraud_prob += 0.10 # was 0.05
if merchant in ['crypto_exchange', 'gaming', 'remittance']: fraud_prob += 0.15 # was 0.07
if merchant in ['financial_services', 'travel']: fraud_prob += 0.05 # new
if amount_vs_avg_ratio > 10: fraud_prob += 0.20 # was 0.12
if amount_vs_avg_ratio > 5: fraud_prob += 0.08 # new
if failed_tx_7d >= 3: fraud_prob += 0.15 # was 0.08
if failed_tx_7d >= 1: fraud_prob += 0.05 # new
if login_attempts_24h >= 5: fraud_prob += 0.15 # was 0.06
if login_attempts_24h >= 3: fraud_prob += 0.05 # new
if distinct_countries >= 4: fraud_prob += 0.15 # was 0.07
if distinct_countries >= 3: fraud_prob += 0.05 # new
if velocity_flag: fraud_prob += 0.10 # was 0.05
if account_age_months < 3: fraud_prob += 0.12 # was 0.06
if account_age_months < 6: fraud_prob += 0.05 # new
if payment in ['crypto', 'BNPL']: fraud_prob += 0.10 # was 0.04
if channel in ['USSD', 'ATM']: fraud_prob += 0.05 # new

```

```

fraud_prob = min(fraud_prob, 0.95)
is_fraud = int(np.random.random() < fraud_prob)

```

```

rows.append({
    'transaction_id': f'TXN{100000 + i}',
    'timestamp': ts.strftime('%Y-%m-%d %H:%M:%S'),
    'customer_id': f'CUST{np.random.randint(1000, 9999)}',
    'country': country,
    'region': region,
    'channel': channel,
    'merchant_category': merchant,
    'payment_method': payment,
    'device_type': device,
    'kyc_tier': kyc,
    'amount_usd': amount,
    'account_age_months': account_age_months,
    'tx_count_30d': tx_count_30d,
    'avg_tx_amount_30d': avg_tx_amount_30d,
    'failed_tx_7d': failed_tx_7d,
    'login_attempts_24h': login_attempts_24h,
    'distinct_countries_30d': distinct_countries,
    'is_new_device': is_new_device,
    'is_night_tx': is_night_tx,
    'is_weekend': is_weekend,
    'amount_vs_avg_ratio': amount_vs_avg_ratio,
    'velocity_flag': velocity_flag,
    'ip_country_mismatch': ip_country_mismatch,
    'is_fraud': is_fraud,
})

```

```

return pd.DataFrame(rows)

```

```

df = generate_transactions(N)
print(f" Dataset generated: {df.shape}")
print(f" Fraud rate: {df['is_fraud'].mean():.2%}")
df.head()

```

Dataset generated: (3000, 24)
 Fraud rate: 28.33%

Out[28]:

	transaction_id	timestamp	customer_id	country	region	channel	merchant_category	payment_method
0	TXN100000	2023-04-13 18:14:00	CUST3558	India	Asia	web	healthcare	debit_card
1	TXN100001	2024-04-20 22:14:00	CUST4943	France	Europe	ATM	utilities	BNPL
2	TXN100002	2024-01-23 10:05:00	CUST1064	Canada	North America	web	education	bank_transfer
3	TXN100003	2024-06-04 17:43:00	CUST4840	Argentina	South America	mobile_app	utilities	wallet
4	TXN100004	2023-01-05 14:13:00	CUST3693	Netherlands	Europe	web	gaming	debit_card

5 rows × 24 columns

Introduce Real-World Imperfections

In [29]:

```
np.random.seed(99)

# 1. Missing values (~3%) across 5 columns
for col in ['avg_tx_amount_30d', 'login_attempts_24h', 'device_type',
            'account_age_months', 'distinct_countries_30d']:
    mask = np.random.random(N) < 0.03
    df.loc[mask, col] = np.nan

# 2. ~30 duplicate rows (simulates double-posting / system retry bugs)
dupe_idx = np.random.choice(df.index, size=30, replace=False)
df = pd.concat([df, df.loc[dupe_idx].copy()], ignore_index=True)

# 3. Inconsistent country name casing (e.g. "NIGERIA", "united states")
bad_idx = np.random.choice(df.index, size=50, replace=False)
df.loc[bad_idx, 'country'] = df.loc[bad_idx, 'country'].str.upper()

# 4. Extreme amount outliers – 8 rows multiplied by 1000 (data-entry errors)
bad_amt_idx = np.random.choice(df.index, size=8, replace=False)
df.loc[bad_amt_idx, 'amount_usd'] = df.loc[bad_amt_idx, 'amount_usd'] * 1000

# 5. Shuffle rows so data is not in generation order
df = df.sample(frac=1, random_state=42).reset_index(drop=True)

print(f"Shape with imperfections: {df.shape}")
print(f"\nMissing values: \n{df.isnull().sum()[df.isnull().sum() > 0]}")
print(f"\nDuplicate rows: {df.duplicated().sum()}")
```

Shape with imperfections: (3030, 24)

Missing values:

device_type	98
account_age_months	86
avg_tx_amount_30d	81
login_attempts_24h	94
distinct_countries_30d	88

dtype: int64

Duplicate rows: 30

In [30]:

```
# Save Raw Dataset

raw_path = p('data', 'raw', 'finshield_transactions_raw.csv')
df.to_csv(raw_path, index=False)
print(f" Raw dataset saved →\n    {raw_path}")
```

Raw dataset saved →

C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\data\raw\finshield_transactions_raw.csv

In []:

DATA CLEANING

Load & Inspect Raw Data

In [31]:

```
raw_path = p('data', 'raw', 'finshield_transactions_raw.csv')
df = pd.read_csv(raw_path)

print("Raw shape:", df.shape)
```

```
print("\nData types:\n", df.dtypes)
print("\nMissing values:\n", df.isnull().sum())
df.head()
```

Raw shape: (3030, 24)

Data types:

transaction_id	object
timestamp	object
customer_id	object
country	object
region	object
channel	object
merchant_category	object
payment_method	object
device_type	object
kyc_tier	object
amount_usd	float64
account_age_months	float64
tx_count_30d	int64
avg_tx_amount_30d	float64
failed_tx_7d	int64
login_attempts_24h	float64
distinct_countries_30d	float64
is_new_device	int64
is_night_tx	int64
is_weekend	int64
amount_vs_avg_ratio	float64
velocity_flag	int64
ip_country_mismatch	int64
is_fraud	int64

dtype: object

Missing values:

transaction_id	0
timestamp	0
customer_id	0
country	0
region	0
channel	0
merchant_category	0
payment_method	0
device_type	98
kyc_tier	0
amount_usd	0
account_age_months	86
tx_count_30d	0
avg_tx_amount_30d	81
failed_tx_7d	0
login_attempts_24h	94
distinct_countries_30d	88
is_new_device	0
is_night_tx	0
is_weekend	0
amount_vs_avg_ratio	0
velocity_flag	0
ip_country_mismatch	0
is_fraud	0

dtype: int64

Out[31]:

	transaction_id	timestamp	customer_id	country	region	channel	merchant_category	payment_method	de
0	TXN101207	2023-06-28 11:03:00	CUST5739	Canada	North America	web	subscription	wallet	
1	TXN100256	2023-01-28 13:59:00	CUST8848	Canada	North America	mobile_app	restaurants	credit_card	
2	TXN102356	2023-12-10 16:30:00	CUST2804	Nigeria	Africa	POS	e-commerce	debit_card	
3	TXN100175	2024-02-20 16:05:00	CUST3971	France	Europe	mobile_app	restaurants	debit_card	
4	TXN100211	2024-03-20 17:34:00	CUST3659	Malaysia	Asia	ATM	e-commerce	debit_card	

5 rows x 24 columns



In []:

Remove Duplicates

```
In [32]: before = len(df)
df = df.drop_duplicates()
print(f" Removed {before - len(df)} duplicate rows. Remaining: {len(df):,}")
```

Removed 30 duplicate rows. Remaining: 3,000

```
In [ ]:
```

Fix Country Name Casing

```
In [33]: df['country'] = df['country'].str.title()
print(f" Country casing standardised. Unique countries: {df['country'].nunique()}")
```

Country casing standardised. Unique countries: 40

```
In [ ]:
```

Handle Missing Values

```
In [34]: # Numerical columns → median imputation (robust to skewed distributions)
num_na_cols = ['avg_tx_amount_30d', 'login_attempts_24h',
               'account_age_months', 'distinct_countries_30d']

for col in num_na_cols:
    median_val = df[col].median()
    df[col] = df[col].fillna(median_val)
    print(f" {col}: filled NaN → median = {median_val:.2f}")

# Categorical column → mode imputation
mode_val = df['device_type'].mode()[0]
df['device_type'] = df['device_type'].fillna(mode_val)
print(f" device_type: filled NaN → mode = '{mode_val}'")

print(f"\n Total remaining nulls: {df.isnull().sum().sum()}")
```

avg_tx_amount_30d: filled NaN → median = 120.20
login_attempts_24h: filled NaN → median = 1.00
account_age_months: filled NaN → median = 60.00
distinct_countries_30d: filled NaN → median = 1.00
device_type: filled NaN → mode = 'Android'

Total remaining nulls: 0

```
In [ ]:
```

Cap Extreme Amount Outliers

```
In [35]: # 3×IQR upper fence catches only true extremes (data-entry errors),
# not legitimate high-value transactions
Q1 = df['amount_usd'].quantile(0.25)
Q3 = df['amount_usd'].quantile(0.75)
upper_fence = Q3 + 3.0 * (Q3 - Q1)

print(f"Max before capping: ${df['amount_usd'].max():>15,.2f}")
df['amount_usd'] = df['amount_usd'].clip(upper=upper_fence)
print(f"Max after capping:  ${df['amount_usd'].max():>15,.2f}")

# Recompute ratio now that amounts are corrected
df['amount_vs_avg_ratio'] = (
    df['amount_usd'] / (df['avg_tx_amount_30d'] + 1e-6)
).round(4)
print(" Outliers capped and ratio recomputed.")
```

Max before capping: \$ 947,280.00
Max after capping: \$ 1,470.86
Outliers capped and ratio recomputed.

```
In [ ]:
```

Parse Timestamps & Extract Time Features

```
In [36]: df['timestamp'] = pd.to_datetime(df['timestamp'])
df['hour'] = df['timestamp'].dt.hour
df['day_of_week'] = df['timestamp'].dt.dayofweek # 0=Monday, 6=Sunday
df['month'] = df['timestamp'].dt.month
df['quarter'] = df['timestamp'].dt.quarter

print(" Time features extracted.")
print(df[['timestamp', 'hour', 'day_of_week', 'month', 'quarter']].head())
```

```

Time features extracted.
      timestamp  hour  day_of_week  month  quarter
0 2023-06-28 11:03:00    11         2      6        2
1 2023-01-28 13:59:00    13         5      1        1
2 2023-12-10 16:30:00    16         6     12        4
3 2024-02-20 16:05:00    16         1      2        1
4 2024-03-20 17:34:00    17         2      3        1

```

In []:

Enforce Data Types & Save Cleaned Data

```

In [37]: int_cols = ['is_new_device', 'is_night_tx', 'is_weekend',
                    'velocity_flag', 'ip_country_mismatch', 'is_fraud']
for col in int_cols:
    df[col] = df[col].astype(int)

print(f" Cleaned shape: {df.shape}")
print(f"   Fraud rate:   {df['is_fraud'].mean():.2%}")

cleaned_path = p('data', 'processed', 'finshield_cleaned.csv')
df.to_csv(cleaned_path, index=False)
print(f"\n Cleaned data saved ->\n   {cleaned_path}")

```

```

Cleaned shape: (3000, 28)
Fraud rate:    28.33%

```

Cleaned data saved ->

C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\data\processed\finshield_cleaned.csv

In []:

PHASE 3 — EXPLORATORY DATA ANALYSIS

Load Cleaned Data for EDA

```

In [2]: cleaned_path = p('data', 'processed', 'finshield_cleaned.csv')
df      = pd.read_csv(cleaned_path)
fraud   = df[df['is_fraud'] == 1]
legit   = df[df['is_fraud'] == 0]

print(f"Total: {len(df):,} | Legitimate: {len(legit):,} | Fraud: {len(fraud):,}")
print(f"Fraud rate: {df['is_fraud'].mean():.2%}")

```

```

Total: 3,000 | Legitimate: 2,150 | Fraud: 850
Fraud rate: 28.33%

```

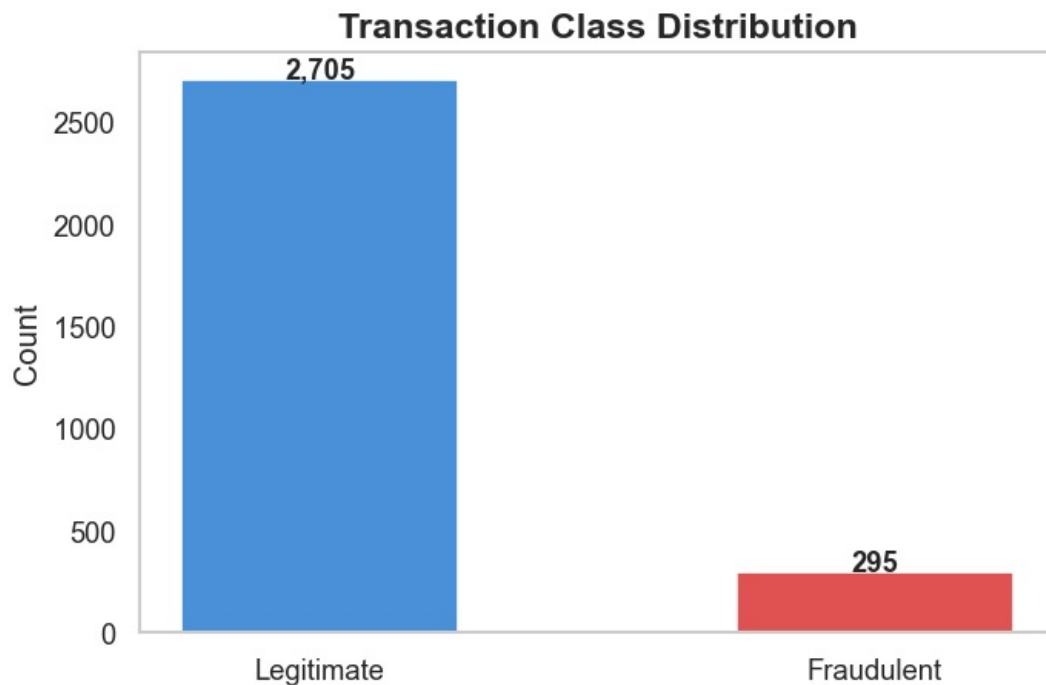
In []:

Class Distribution

```

In [18]: fig, ax = plt.subplots(figsize=(6, 4))
counts = df['is_fraud'].value_counts()
colors = ['#4A90D9', '#E05252']
bars   = ax.bar(['Legitimate', 'Fraudulent'], counts.values, color=colors, width=0.5)
ax.set_title('Transaction Class Distribution', fontsize=14, fontweight='bold')
ax.set_ylabel('Count')
for bar, val in zip(bars, counts.values):
    ax.text(bar.get_x() + bar.get_width() / 2, val + 8,
            f'{val:,}', ha='center', fontweight='bold', fontsize=11)
ax.grid(False)
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '01_class_distribution.png'), bbox_inches='tight')
plt.show()

```

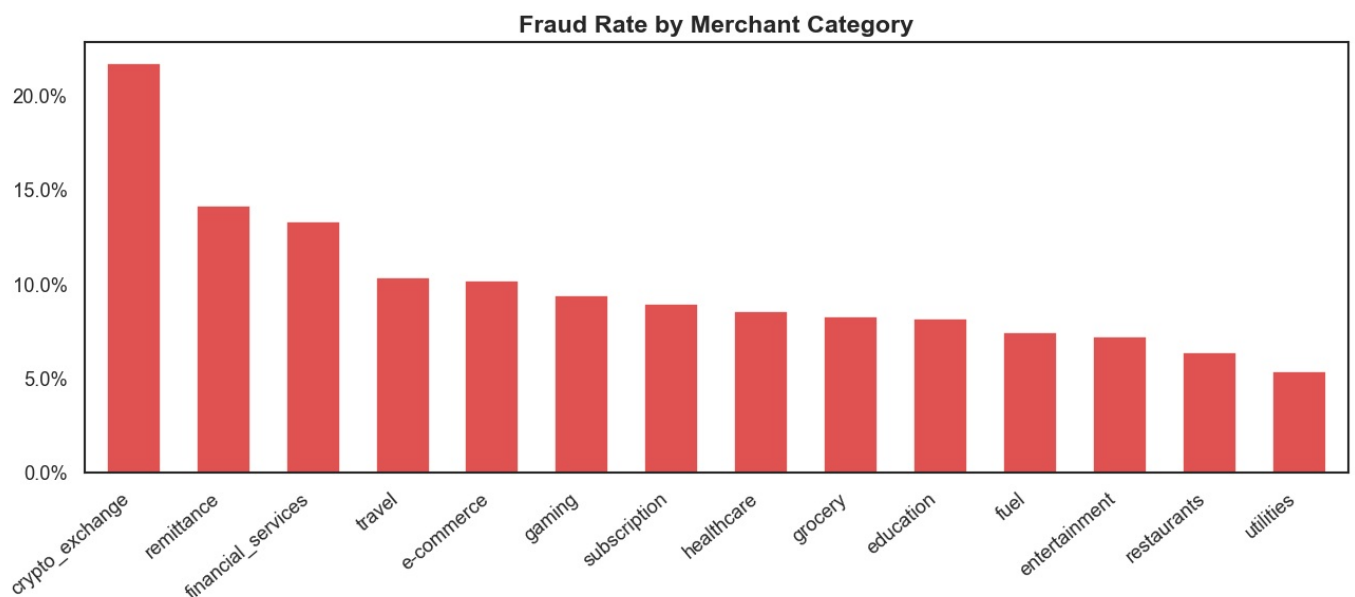



The dataset contains 2,705 legitimate transactions (90.2%) and 295 fraudulent ones (9.8%), confirming a significant class imbalance typical of real-world fraud data.

Fraud Rate by Merchant Category

```
In [20]: fraud_by_merchant = (df.groupby('merchant_category')['is_fraud']
                                .mean()
                                .sort_values(ascending=False))

fig, ax = plt.subplots(figsize=(11, 5))
fraud_by_merchant.plot(kind='bar', ax=ax, color='#E05252', width=0.6)
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Fraud Rate by Merchant Category', fontsize=14, fontweight='bold')
ax.set_xlabel('')
ax.set_xticklabels(ax.get_xticklabels(), rotation=40, ha='right')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '02_fraud_by_merchant.png'), bbox_inches='tight')
plt.show()
```



- Crypto exchange has the highest fraud rate at 22%, more than double the next category, driven by its anonymity and irreversible transactions.
- Remittance (14%) and financial services (13%) rank next, confirming high-value and cross-border merchants are prime fraud targets.
- Utilities and restaurants record the lowest rates (5-6%), as their low-value, predictable transactions are less attractive to fraudsters.

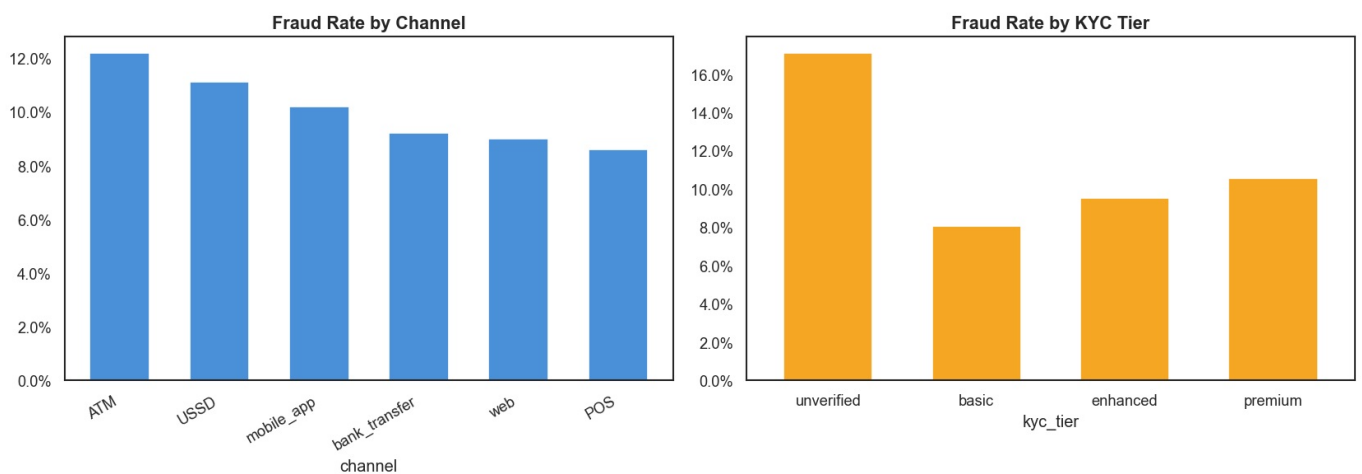
Fraud Rate by Channel and KYC Tier

```
In [21]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Channel
fraud_by_channel = (df.groupby('channel')['is_fraud']
                    .mean()
                    .sort_values(ascending=False))
fraud_by_channel.plot(kind='bar', ax=axes[0], color='#4A90D9', width=0.6)
axes[0].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[0].set_title('Fraud Rate by Channel', fontsize=13, fontweight='bold')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=30, ha='right')

# KYC tier – ordered from most to least risky
kyc_order = ['unverified', 'basic', 'enhanced', 'premium']
fraud_by_kyc = df.groupby('kyc_tier')['is_fraud'].mean().reindex(kyc_order)
fraud_by_kyc.plot(kind='bar', ax=axes[1], color='#F5A623', width=0.6)
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
axes[1].set_title('Fraud Rate by KYC Tier', fontsize=13, fontweight='bold')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=0)

plt.tight_layout()
plt.savefig(p('outputs', 'figures', '03_fraud_by_channel_kyc.png'), bbox_inches='tight')
plt.show()
```



Fraud Rate by Channel and KYC Tier Channel:

- ATM (12%) and USSD (11%) record the highest fraud rates, as both channels offer limited real-time authentication and are widely exploited in card skimming and SIM swap attacks.
- POS records the lowest rate (8.6%), likely due to physical card presence and in-person verification reducing remote fraud opportunities.

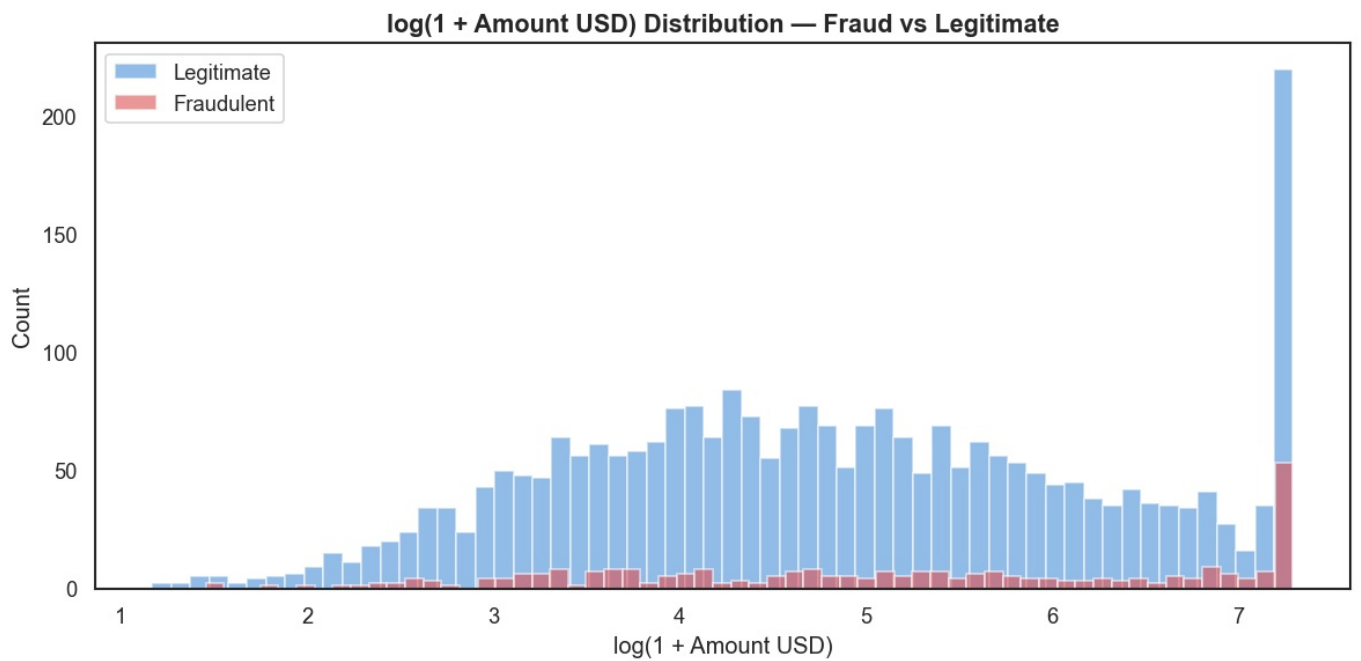
KYC Tier:

- Unverified accounts have a strikingly high fraud rate of 17%, more than double that of basic-tier accounts, confirming that identity verification is a powerful fraud deterrent.
- An unexpected pattern emerges where premium accounts (10.6%) show higher fraud than enhanced (9.6%), suggesting high-value verified accounts are actively targeted for takeover by advanced fraudsters.

In []:

Amount Distribution: Fraud vs Legitimate

```
In [22]: fig, ax = plt.subplots(figsize=(10, 5))
ax.hist(np.log1p(legit['amount_usd']), bins=60, alpha=0.6,
        label='Legitimate', color='#4A90D9')
ax.hist(np.log1p(fraud['amount_usd']), bins=60, alpha=0.6,
        label='Fraudulent', color='#E05252')
ax.set_title('log(1 + Amount USD) Distribution – Fraud vs Legitimate',
            fontsize=13, fontweight='bold')
ax.set_xlabel('log(1 + Amount USD)')
ax.set_ylabel('Count')
ax.legend()
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '04_amount_distribution.png'), bbox_inches='tight')
plt.show()
```



Fraudulent transactions maintain a consistently low but persistent presence across all amount ranges, confirming that fraudsters operate at every spending level

In []:

Correlation Heatmap

In [7]:

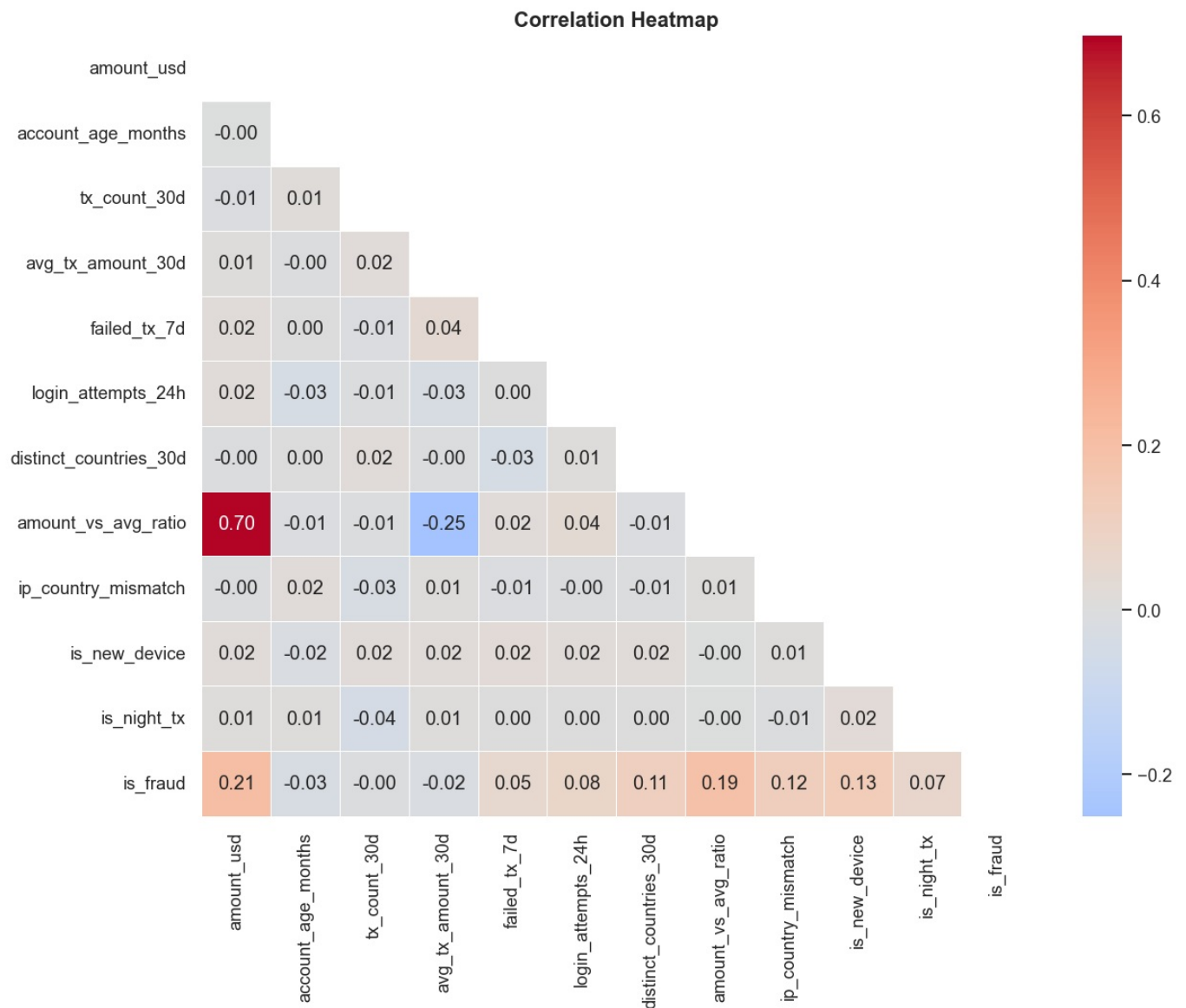
```
num_features = [
    'amount_usd', 'account_age_months', 'tx_count_30d', 'avg_tx_amount_30d',
    'failed_tx_7d', 'login_attempts_24h', 'distinct_countries_30d',
    'amount_vs_avg_ratio', 'ip_country_mismatch', 'is_new_device',
    'is_night_tx', 'is_fraud' # velocity_flag removed
]
```

In []:

In [9]:

```
num_features = [
    'amount_usd', 'account_age_months', 'tx_count_30d', 'avg_tx_amount_30d',
    'failed_tx_7d', 'login_attempts_24h', 'distinct_countries_30d',
    'amount_vs_avg_ratio', 'ip_country_mismatch', 'is_new_device',
    'is_night_tx', 'is_fraud'
]
corr = df[num_features].corr()
mask = np.triu(np.ones_like(corr, dtype=bool))

fig, ax = plt.subplots(figsize=(11, 9))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, linewidths=0.5, ax=ax)
ax.set_title('Correlation Heatmap', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '05_correlation_heatmap.png'), bbox_inches='tight')
plt.show()
```



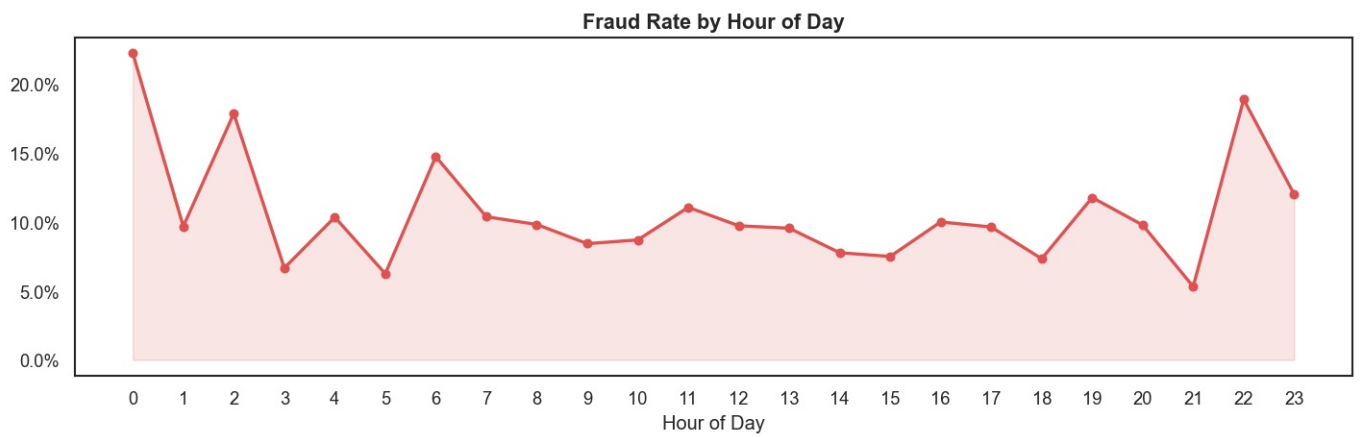
- **amount_usd** shows the strongest positive correlation with **is_fraud (0.21)**, meaning higher transaction amounts are the single biggest linear fraud indicator in the dataset.
- **amount_vs_avg_ratio** correlates strongly with **amount_usd (0.70)** — expected, since the ratio is derived from amount — while its correlation with **is_fraud (0.19)** confirms that transacting far above one's own average is a reliable fraud signal.
- **ip_country_mismatch (0.12)**, **is_new_device (0.13)**, and **distinct_countries_30d (0.11)** all show meaningful positive correlations with fraud, validating their inclusion as key risk features in the model.

In []:

Fraud Rate by Hour of Day

```
In [24]: fraud_by_hour = df.groupby('hour')['is_fraud'].mean()

fig, ax = plt.subplots(figsize=(12, 4))
ax.plot(fraud_by_hour.index, fraud_by_hour.values,
        marker='o', color='#E05252', lw=2, markersize=5)
ax.fill_between(fraud_by_hour.index, fraud_by_hour.values,
               alpha=0.15, color='#E05252')
ax.yaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Fraud Rate by Hour of Day', fontsize=13, fontweight='bold')
ax.set_xlabel('Hour of Day')
ax.set_xticks(range(0, 24))
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '06_fraud_by_hour.png'), bbox_inches='tight')
plt.show()
```



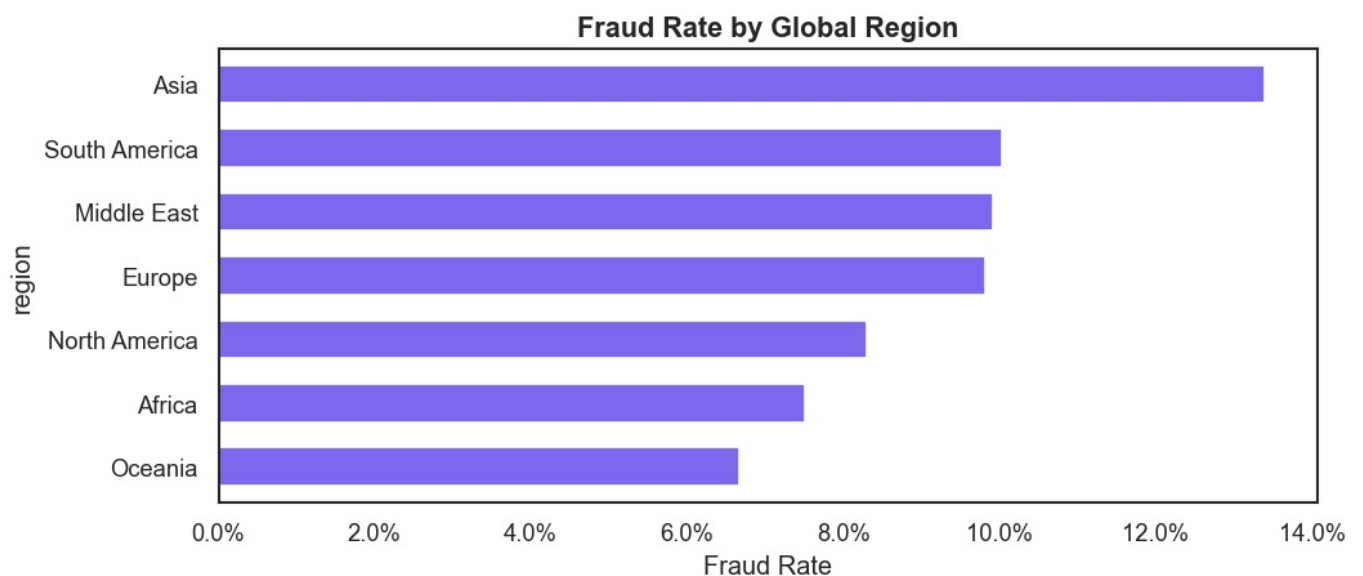
- Midnight (00:00) records the highest fraud rate at 22%, with 2am (18%) and 6am (15%) also spiking sharply, confirming that late-night and early-morning hours are the most exploited window for fraudulent activity.
- Fraud rates drop and stabilise between 8am and 5pm (8–11%), reflecting normal daytime transaction behaviour where higher monitoring occurs.
- A secondary spike emerges at 10pm–11pm (19%), suggesting fraudsters operate in two concentrated windows, deep night and late evening.

In []:

Fraud Rate by Global Region

```
In [28]: fraud_by_region = (df.groupby('region')['is_fraud']
                             .mean()
                             .sort_values(ascending=True))

fig, ax = plt.subplots(figsize=(9, 4))
fraud_by_region.plot(kind='barh', ax=ax, color='#7B68EE', width=0.6)
ax.xaxis.set_major_formatter(mtick.PercentFormatter(1.0))
ax.set_title('Fraud Rate by Global Region', fontsize=13, fontweight='bold')
ax.set_xlabel('Fraud Rate')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '07_fraud_by_region.png'), bbox_inches='tight')
plt.show()
```



- Asia records the highest fraud rate at ~13.5%, significantly ahead of all other regions, reflecting its large transaction volumes

In []:

PHASE 4 — FEATURE ENGINEERING

load cleaned data fresh

```
In [38]: cleaned_path = p('data', 'processed', 'finshield_cleaned.csv')
df = pd.read_csv(cleaned_path)

print(f"Cleaned data loaded: {df.shape}")
print(f"Fraud rate: {df['is_fraud'].mean():.2%}")
print(f"\nDtypes:\n{df.dtypes.value_counts()}")
```

Cleaned data loaded: (3000, 28)
Fraud rate: 28.33%

Dtypes:
int64 12
object 10
float64 6
Name: count, dtype: int64

In []:

feature engineering in memory

```
In [39]: fe = df.copy()

# Log transforms
fe['log_amount']      = np.log1p(fe['amount_usd'])
fe['log_avg_tx_amt']  = np.log1p(fe['avg_tx_amount_30d'])
fe['log_amount_ratio'] = np.log1p(fe['amount_vs_avg_ratio'])

# Composite risk score
fe['risk_score'] = (
    fe['ip_country_mismatch'] * 2.0 +
    fe['is_new_device'] * 1.5 +
    fe['is_night_tx'] * 1.0 +
    fe['velocity_flag'] * 1.5 +
    (fe['failed_tx_7d'] >= 3).astype(int) * 2.0 +
    (fe['login_attempts_24h'] >= 5).astype(int) * 1.5 +
    (fe['distinct_countries_30d'] >= 4).astype(int) * 2.0 +
    (fe['account_age_months'] < 3).astype(int) * 1.0
)

# KYC ordinal encoding
kyc_map = {'unverified': 0, 'basic': 1, 'enhanced': 2, 'premium': 3}
fe['kyc_encoded'] = fe['kyc_tier'].map(kyc_map)

# One-hot encode categoricals
ohe_cols = ['channel', 'merchant_category', 'payment_method', 'device_type', 'region']
fe = pd.get_dummies(fe, columns=ohe_cols, drop_first=True)

# Convert ALL bool columns to int
bool_cols = fe.select_dtypes(include='bool').columns.tolist()
fe[bool_cols] = fe[bool_cols].astype(int)

# Drop irrelevant columns
drop_cols = [
    'transaction_id', 'customer_id', 'timestamp', 'country', 'kyc_tier',
    'amount_usd', 'avg_tx_amount_30d', 'amount_vs_avg_ratio'
]
fe = fe.drop(columns=drop_cols, errors='ignore')

print(f" Feature set shape: {fe.shape}")
print(f"  Fraud rate: {fe['is_fraud'].mean():.2%}")
print(f"\nDtype summary:\n{fe.dtypes.value_counts()}")
print(f"\nAny nulls: {fe.isnull().sum().sum()}")
print(f"Any inf: {np.isinf(fe.select_dtypes(include=np.number)).sum().sum()}")
print(f"\nTarget distribution:\n{fe['is_fraud'].value_counts()}")
```

Feature set shape: (3000, 54)
Fraud rate: 28.33%

Dtype summary:
int64 47
float64 7
Name: count, dtype: int64

Any nulls: 0
Any inf: 0

Target distribution:
is_fraud
0 2150
1 850
Name: count, dtype: int64

In []:

signal check — fraud vs legit means

```
In [40]: # Check that key features actually separate fraud from legit
# If these show no difference, the cleaned data itself has an issue

fraud_mask = fe['is_fraud'] == 1
legit_mask = fe['is_fraud'] == 0

check_cols = ['risk_score', 'log_amount_ratio', 'log_amount',
              'ip_country_mismatch', 'is_new_device', 'kyc_encoded']

print("Feature means — Fraud vs Legitimate:")
print("-" * 45)
for col in check_cols:
    if col in fe.columns:
        fraud_mean = fe.loc[fraud_mask, col].mean()
        legit_mean = fe.loc[legit_mask, col].mean()
        diff = fraud_mean - legit_mean
        print(f" {col:<25} fraud={fraud_mean:.3f} legit={legit_mean:.3f} diff={diff:+.3f}")
```

Feature means — Fraud vs Legitimate:

risk_score	fraud=1.232	legit=0.667	diff=+0.566
log_amount_ratio	fraud=1.266	legit=0.898	diff=+0.369
log_amount	fraud=5.244	legit=4.748	diff=+0.495
ip_country_mismatch	fraud=0.213	legit=0.121	diff=+0.092
is_new_device	fraud=0.264	legit=0.154	diff=+0.110
kyc_encoded	fraud=1.479	legit=1.666	diff=-0.187

In []:

train/test split

```
In [41]: X = fe.drop(columns=['is_fraud'])
y = fe['is_fraud']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

print(f"Train: {X_train.shape} | Test: {X_test.shape}")
print(f"Train fraud rate: {y_train.mean():.2%} | Test fraud rate: {y_test.mean():.2%}")
```

Train: (2400, 53) | Test: (600, 53)

Train fraud rate: 28.33% | Test fraud rate: 28.33%

In []:

PHASE 5 — MODEL TRAINING & EVALUATION

SMOTE (Synthetic Minority Over-sampling Technique)

Handling class imbalance in the training data.

```
In [42]: smote = SMOTE(random_state=42, k_neighbors=5)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)

print(f" After SMOTE: {X_train_res.shape}")
print(f" Class counts: {pd.Series(y_train_res).value_counts().to_dict()}")
```

After SMOTE: (3440, 53)

Class counts: {0: 1720, 1: 1720}

In []:

Scale + Train All Models

```
In [43]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_res)
X_test_scaled = scaler.transform(X_test)

joblib.dump(scaler, p('models', 'scaler.pkl'))

# Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42, class_weight='balanced')
lr.fit(X_train_scaled, y_train_res)
print(" Logistic Regression trained.")
```

```
# Random Forest
rf = RandomForestClassifier(
    n_estimators=200, max_depth=12, min_samples_leaf=5,
    class_weight='balanced', random_state=42, n_jobs=-1
)
rf.fit(X_train_res, y_train_res)
print(" Random Forest trained.")

# XGBoost
scale_pos = (y_train_res == 0).sum() / (y_train_res == 1).sum()
xgb = XGBClassifier(
    n_estimators=300, max_depth=6, learning_rate=0.05,
    subsample=0.8, colsample_bytree=0.8,
    scale_pos_weight=scale_pos,
    eval_metric='aucpr',
    random_state=42, n_jobs=-1
)
xgb.fit(X_train_res, y_train_res,
        eval_set=[(X_test, y_test)], verbose=False)
print(" XGBoost trained.")

Logistic Regression trained.
Random Forest trained.
XGBoost trained.
```

In []:

Compare Models

```
In [44]: models = {
    'Logistic Regression': (lr, X_test_scaled),
    'Random Forest':      (rf, X_test),
    'XGBoost':             (xgb, X_test),
}

results = {}
for name, (model, X_eval) in models.items():
    y_pred = model.predict(X_eval)
    y_prob = model.predict_proba(X_eval)[:, 1]
    results[name] = {
        'ROC-AUC':      round(roc_auc_score(y_test, y_prob), 4),
        'Avg Precision': round(average_precision_score(y_test, y_prob), 4),
        'F1 (Fraud)':   round(f1_score(y_test, y_pred), 4),
    }

comparison_df = pd.DataFrame(results).T
print("=" * 52)
print("          MODEL COMPARISON")
print("=" * 52)
print(comparison_df.to_string())

comparison_df.to_csv(p('outputs', 'reports', 'model_comparison.csv'))
```

MODEL COMPARISON			
	ROC-AUC	Avg Precision	F1 (Fraud)
Logistic Regression	0.6990	0.4707	0.3383
Random Forest	0.7050	0.4700	0.3985
XGBoost	0.7115	0.4783	0.4270

- XGBoost outperforms all models across every metric: **ROC-AUC (0.712)**, **Avg Precision (0.478)**, and **F1 (0.427)** , making it the clear choice as the production model for the FinShield AI fraud detection system.

XGBoost Classification Report

```
In [45]: y_pred_xgb = xgb.predict(X_test)
print("=" * 50)
print("  XGBoost - Full Classification Report")
print("=" * 50)
print(classification_report(y_test, y_pred_xgb,
                           target_names=['Legitimate', 'Fraud']))
```


XGBoost – Full Classification Report				
	precision	recall	f1-score	support
Legitimate	0.78	0.88	0.82	430
Fraud	0.54	0.35	0.43	170
accuracy			0.73	600
macro avg	0.66	0.62	0.63	600
weighted avg	0.71	0.73	0.71	600

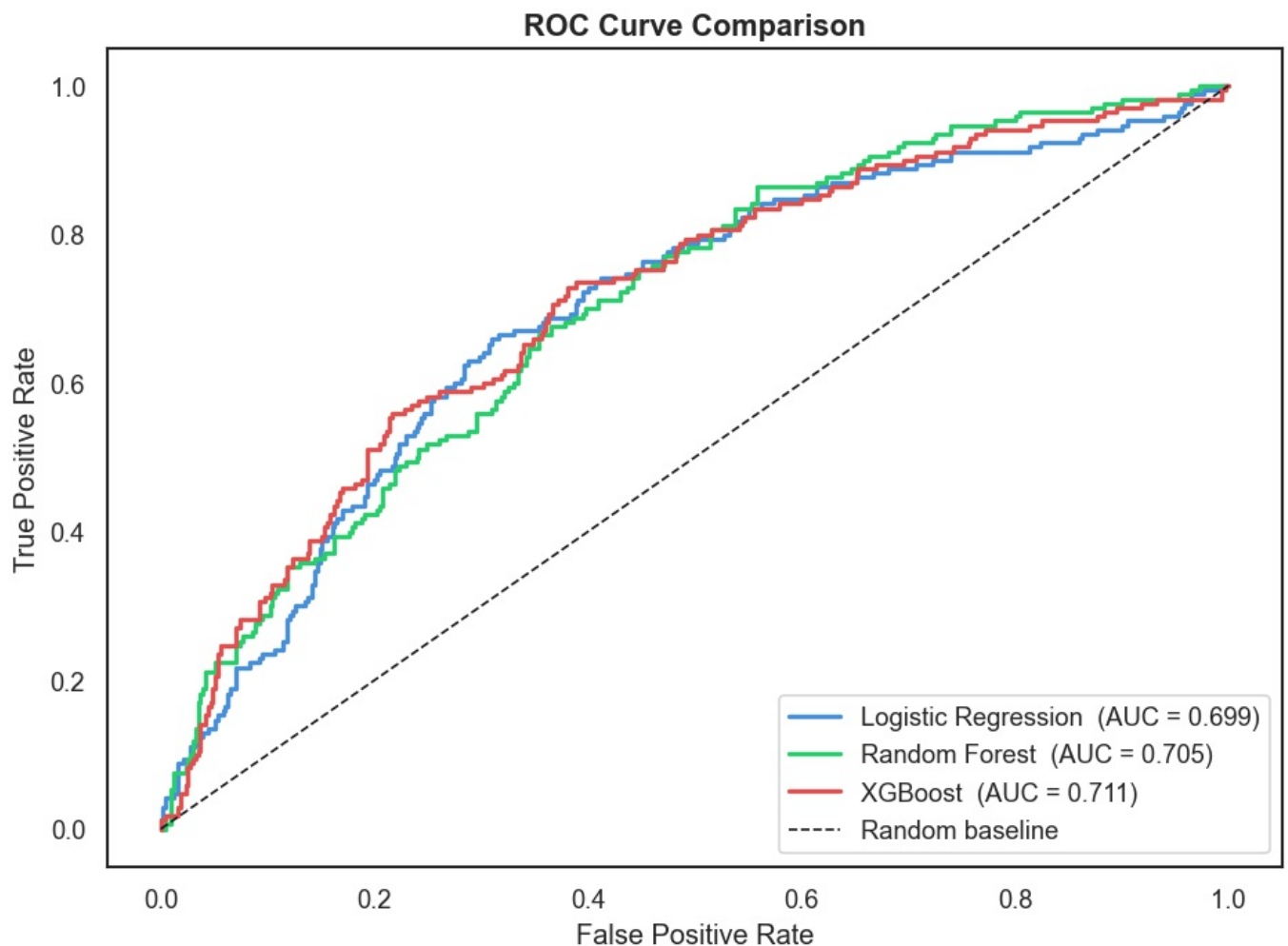
In []:

ROC Curve Comparison

```
In [47]: fig, ax = plt.subplots(figsize=(8, 6))
colors = ['#4A90D9', '#2ECC71', '#E05252']

for (name, (model, X_eval)), color in zip(models.items(), colors):
    y_prob = model.predict_proba(X_eval)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc = roc_auc_score(y_test, y_prob)
    ax.plot(fpr, tpr, lw=2, color=color, label=f'{name} (AUC = {auc:.3f})')

ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random baseline')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC Curve Comparison', fontsize=13, fontweight='bold')
ax.legend(loc='lower right')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '08_roc_comparison.png'), bbox_inches='tight') # ← fixed
plt.show()
```

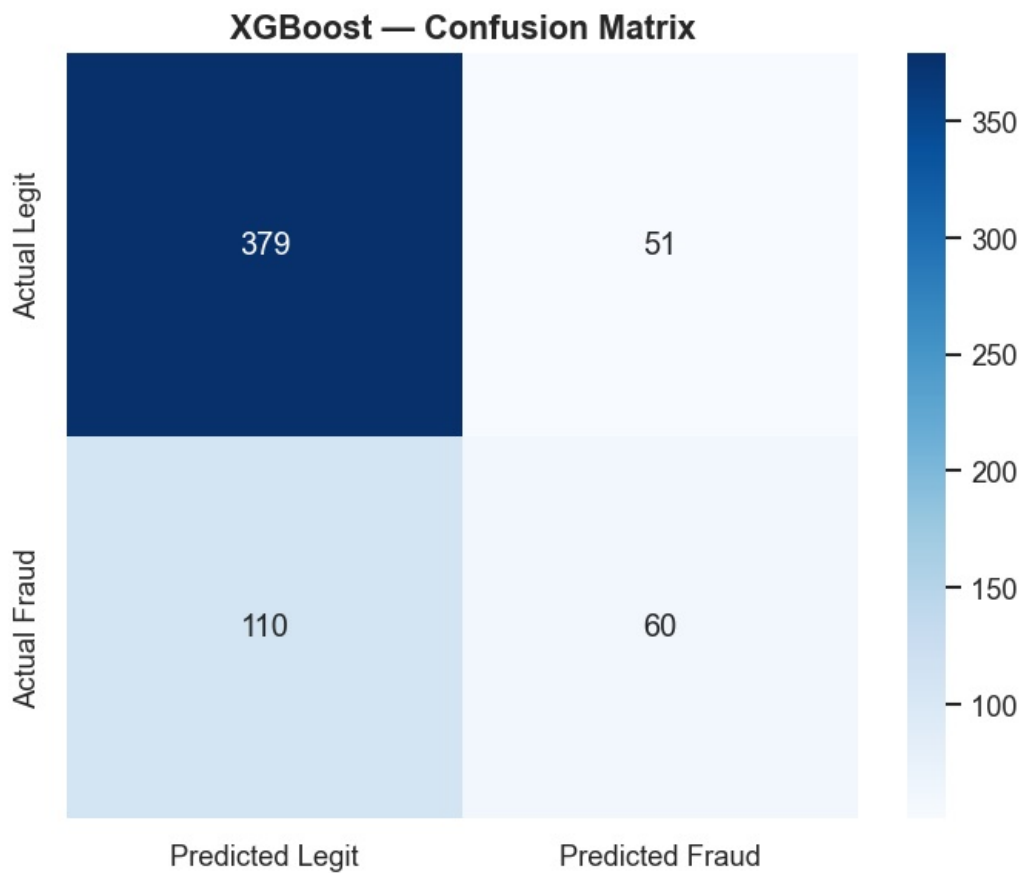


In []:

Confusion Matrix (XGBoost)

```
In [48]: cm = confusion_matrix(y_test, y_pred_xgb)
```

```
fig, ax = plt.subplots(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
            xticklabels=['Predicted Legit', 'Predicted Fraud'],
            yticklabels=['Actual Legit', 'Actual Fraud'])
ax.set_title('XGBoost — Confusion Matrix', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '09_confusion_matrix.png'), bbox_inches='tight')
plt.show()
```

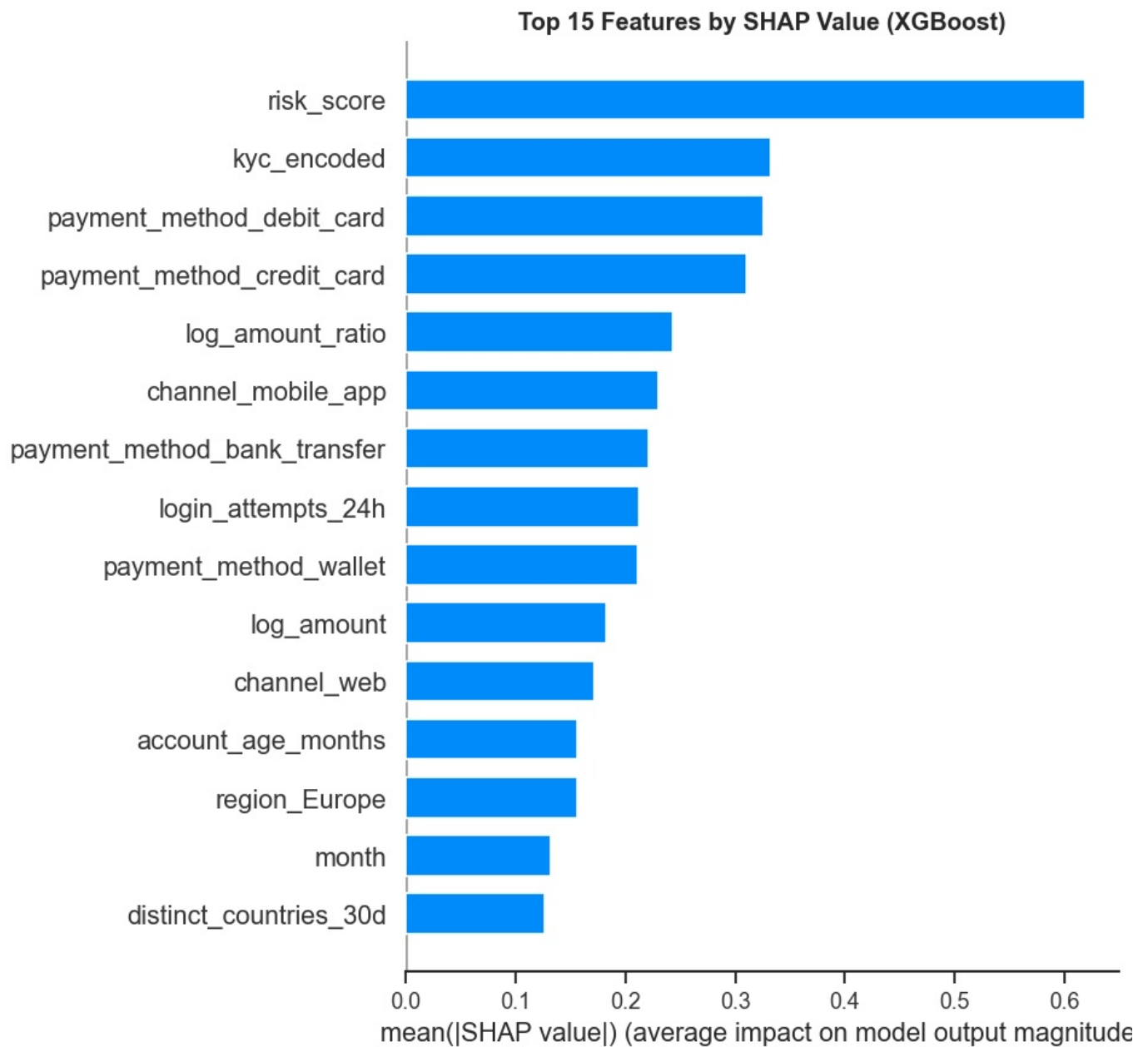


- The model correctly identified 379 legitimate transactions and 60 fraudulent ones, demonstrating reasonable ability to detect fraud while keeping false alarms manageable with only 51 legitimate transactions wrongly flagged.

SHAP Feature Importance

```
In [49]: explainer = shap.TreeExplainer(xgb)
shap_values = explainer.shap_values(X_test)

shap.summary_plot(shap_values, X_test, plot_type='bar',
                 max_display=15, show=False)
plt.title('Top 15 Features by SHAP Value (XGBoost)', fontsize=12, fontweight='bold')
plt.tight_layout()
plt.savefig(p('outputs', 'figures', '10_shap_importance.png'), bbox_inches='tight')
plt.show()
```



In []:

Save Best Model & Feature Columns

In [51]:

```
model_path = p('models', 'xgb_fraud_model.pkl')
features_json = p('models', 'feature_columns.json')

joblib.dump(xgb, model_path)

with open(features_json, 'w') as f:
    json.dump(list(X.columns), f)

print(f" Model saved    → {model_path}")
print(f" Features saved → {features_json}")
print(f" Scaler saved    → {p('models', 'scaler.pkl')}")
print("\n All model artefacts ready.")
```

Model saved → C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\models\xgb_fraud_model.pkl
Features saved → C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\models\feature_columns.json
Scaler saved → C:\Users\DELL\Desktop\Data Analytics Projects\finshield_ai\models\scaler.pkl

All model artefacts ready.

In []:

PHASE 6 — STREAMLIT APP

Preparing Deployment Folder

In [2]: `import os`

```
import shutil

username = os.getlogin() # Gets the currently logged-in Windows user
PROJECT_ROOT = rf"C:\Users\{username}\Desktop\Data Analytics Projects\finshield_ai"

# Deployment folder - The 'Destination': A clean folder on the Desktop dedicated to the live app/deployment
DEPLOY_ROOT = rf"C:\Users\{username}\Desktop\finshield_deploy"

# Create deployment folder structure
os.makedirs(os.path.join(DEPLOY_ROOT, 'models'), exist_ok=True)
print(f" Deployment folder created at:\n {DEPLOY_ROOT}")

# Copy model artefacts ( files generated during training that are needed for future prediction )
files_to_copy = [
    (p('models', 'xgb_fraud_model.pkl'), os.path.join(DEPLOY_ROOT, 'models', 'xgb_fraud_model.pkl')),
    (p('models', 'scaler.pkl'), os.path.join(DEPLOY_ROOT, 'models', 'scaler.pkl')),
    # A JSON list of column names (ensures the model gets features in the right order)
    (p('models', 'feature_columns.json'), os.path.join(DEPLOY_ROOT, 'models', 'feature_columns.json')),
]

for src, dst in files_to_copy:
    shutil.copy2(src, dst)
    print(f" Copied: {os.path.basename(src)}")

print("\n Model artefacts ready for deployment.")
```

```
Deployment folder created at:
C:\Users\DELL\Desktop\finshield_deploy
Copied: xgb_fraud_model.pkl
Copied: scaler.pkl
Copied: feature_columns.json
```

```
Model artefacts ready for deployment.
```

In []:

Write app.py for Cloud (Relative Paths)

In [3]:

```
app_code = """
FinShield AI – Real-Time Fraud Detection App
Deployed via Streamlit Community Cloud
"""

import os
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import streamlit as st

st.set_page_config(
    page_title="FinShield AI | Fraud Detector",
    page_icon="🛡️",
    layout="wide"
)

@st.cache_resource
def load_artifacts():
    base = os.path.dirname(os.path.abspath(__file__))
    model_path = os.path.join(base, "models", "xgb_fraud_model.pkl")
    scaler_path = os.path.join(base, "models", "scaler.pkl")
    feature_path = os.path.join(base, "models", "feature_columns.json")

    model = joblib.load(model_path)
    scaler = joblib.load(scaler_path)
    with open(feature_path) as f:
        feature_cols = json.load(f)
    return model, scaler, feature_cols

model, scaler, feature_cols = load_artifacts()

# — Header —————
st.title("🛡️ FinShield AI – Fraud Detection System")
st.markdown("***NovaPay International · Global Transaction Intelligence Platform**")
st.markdown(
    "Enter transaction details below and click **Analyse Transaction** "
    "to get an instant fraud risk assessment."
)
st.divider()

# — Input form —————
st.subheader(" Transaction Details")
```

```

c1, c2, c3 = st.columns(3)

with c1:
    st.markdown("***Transaction Info**")
    amount_usd = st.number_input("Transaction Amount (USD)", 1.0, 100000.0, 250.0, step=50.0)
    channel = st.selectbox("Channel", [
        "mobile_app", "web", "POS", "ATM", "USSD", "bank_transfer"])
    merchant_category = st.selectbox("Merchant Category", [
        "e-commerce", "grocery", "utilities", "travel", "entertainment",
        "restaurants", "healthcare", "financial_services",
        "crypto_exchange", "gaming", "remittance", "fuel",
        "education", "subscription"])
    payment_method = st.selectbox("Payment Method", [
        "debit_card", "credit_card", "wallet",
        "bank_transfer", "crypto", "BNPL"])

with c2:
    st.markdown("***Account Info**")
    device_type = st.selectbox("Device Type", [
        "Android", "iOS", "Windows", "macOS", "Linux", "Unknown"])
    kyc_tier = st.selectbox("KYC Tier", [
        "unverified", "basic", "enhanced", "premium"])
    region = st.selectbox("Region", [
        "North America", "Europe", "Asia", "Africa",
        "South America", "Middle East", "Oceania"])
    account_age_months = st.slider("Account Age (months)", 1, 120, 12)

with c3:
    st.markdown("***Behavioural History**")
    tx_count_30d = st.slider("Transactions last 30 days", 0, 100, 15)
    avg_tx_amount_30d = st.number_input("Avg Transaction Amount 30d (USD)",
        1.0, 50000.0, 200.0)
    failed_tx_7d = st.slider("Failed Transactions (last 7 days)", 0, 20, 0)
    login_attempts_24h = st.slider("Login Attempts (last 24h)", 0, 20, 1)

st.divider()
c4, c5 = st.columns(2)

with c4:
    st.markdown("***Time & Location**")
    distinct_countries_30d = st.slider("Distinct Countries Transacted (30 days)", 1, 10, 1)
    hour = st.slider("Transaction Hour (0-23)", 0, 23, 14)
    day_of_week = st.slider("Day of Week (0=Mon, 6=Sun)", 0, 6, 2)
    month = st.slider("Month", 1, 12, 6)

with c5:
    st.markdown("***Risk Flags**")
    is_new_device = st.checkbox("New / Unrecognised Device")
    is_night_tx = st.checkbox("Night Transaction (00:00-05:59)")
    is_weekend = st.checkbox("Weekend Transaction")
    ip_country_mismatch = st.checkbox("IP Address / Country Mismatch")
    velocity_flag = st.checkbox("High Velocity Flag (>40 txns / 30 days)")

# — Feature builder —————
def build_input_row():
    kyc_map = {"unverified": 0, "basic": 1, "enhanced": 2, "premium": 3}
    log_ratio = np.log1p(amount_usd / (avg_tx_amount_30d + 1e-6))

    risk_score = (
        int(ip_country_mismatch) * 2.0 +
        int(is_new_device) * 1.5 +
        int(is_night_tx) * 1.0 +
        int(velocity_flag) * 1.5 +
        int(failed_tx_7d >= 3) * 2.0 +
        int(login_attempts_24h >= 5) * 1.5 +
        int(distinct_countries_30d >= 4) * 2.0 +
        int(account_age_months < 3) * 1.0
    )

    base = {
        "account_age_months": account_age_months,
        "tx_count_30d": tx_count_30d,
        "failed_tx_7d": failed_tx_7d,
        "login_attempts_24h": login_attempts_24h,
        "distinct_countries_30d": distinct_countries_30d,
        "is_new_device": int(is_new_device),
        "is_night_tx": int(is_night_tx),
        "is_weekend": int(is_weekend),
        "velocity_flag": int(velocity_flag),
        "ip_country_mismatch": int(ip_country_mismatch),
        "hour": hour,
        "day_of_week": day_of_week,
        "month": month,
    }

```

```

        "quarter":            (month - 1) // 3 + 1,
        "log_amount":        np.log1p(amount_usd),
        "log_avg_tx_amt":    np.log1p(avg_tx_amount_30d),
        "log_amount_ratio":  log_ratio,
        "risk_score":        risk_score,
        "kyc_encoded":       kyc_map[kyc_tier],
    }

    row = pd.DataFrame([base])

    cat_inputs = {
        "channel": channel, "merchant_category": merchant_category,
        "payment_method": payment_method, "device_type": device_type,
        "region": region,
    }
    for cat, val in cat_inputs.items():
        col_name = f"{cat}_{val}"
        for fc in feature_cols:
            if fc.startswith(cat + "_") and fc not in row.columns:
                row[fc] = 0
        if col_name in feature_cols:
            row[col_name] = 1

    for fc in feature_cols:
        if fc not in row.columns:
            row[fc] = 0

    return row[feature_cols]

# — Prediction button —————
if st.button(" Analyse Transaction", type="primary", use_container_width=True):
    row = build_input_row()
    prob = model.predict_proba(row)[0][1]
    pred = int(prob >= 0.50)

    st.divider()
    st.subheader(" Fraud Risk Assessment")
    res_col, gauge_col = st.columns([1, 2])

    with res_col:
        if pred == 1:
            st.error(" HIGH RISK – FRAUD LIKELY")
        else:
            st.success(" LOW RISK – TRANSACTION CLEAN")
            st.metric("Fraud Probability", f"{prob:.1%}")
            st.metric("Decision Threshold", "50%")
            st.metric("Model", "XGBoost")

    with gauge_col:
        fig, ax = plt.subplots(figsize=(6, 1.8))
        bar_color = "#E05252" if pred == 1 else "#2ECC71"
        ax.barh(["Risk Level"], [prob], color=bar_color, height=0.45)
        ax.barh(["Risk Level"], [1 - prob], left=prob, color="#EEEEEE", height=0.45)
        ax.axvline(0.5, color="orange", lw=2.5, linestyle="--", label="Threshold (0.5)")
        ax.set_xlim(0, 1)
        ax.set_xlabel("Fraud Probability")
        ax.legend(loc="upper right", fontsize=9)
        ax.set_title(f"Fraud Score: {prob:.1%}", fontweight="bold")
        st.pyplot(fig)

    st.divider()
    st.markdown("***What the score means:**")
    if prob < 0.30:
        st.info(" Very low risk. Transaction profile is consistent with normal behaviour.")
    elif prob < 0.50:
        st.warning(" Moderate risk. Some unusual signals present but below fraud threshold.")
    elif prob < 0.75:
        st.error(" High risk. Multiple fraud indicators detected. Review recommended.")
    else:
        st.error(" Very high risk. Strong fraud signals across multiple dimensions. Block or escalate.")

    st.caption("Model: XGBoost · FinShield Synthetic v1.0 · Threshold: 0.50")
...

deploy_app_path = os.path.join(DEPLOY_ROOT, 'app.py')
with open(deploy_app_path, 'w', encoding='utf-8') as f:
    f.write(app_code)
print(f" app.py written →\n    {deploy_app_path}")

```

app.py written →
C:\Users\DELL\Desktop\finshield_deploy\app.py

In []:

In [6]: **import** os

```
username    = os.getlogin()
DEPLOY_ROOT = rf"C:\Users\{username}\Desktop\finshield_deploy"

app_path = os.path.join(DEPLOY_ROOT, 'app.py')

# Read current app.py
with open(app_path, 'r', encoding='utf-8') as f:
    content = f.read()

# Change AI nomenclature
content = content.replace('NovaPay International', 'Zenvra International')
content = content.replace('NovaPay', 'Zenvra International')

# Write back
with open(app_path, 'w', encoding='utf-8') as f:
    f.write(content)

print(" app.py updated – NovaPay replaced with Zenvra International")
print(f"    File: {app_path}")

# Verify the change
with open(app_path, 'r', encoding='utf-8') as f:
    for i, line in enumerate(f, 1):
        if 'Zenvra' in line or 'NovaPay' in line:
            print(f"    Line {i}: {line.strip()}")

app.py updated – NovaPay replaced with Zenvra International
File: C:\Users\DELL\Desktop\finshield_deploy\app.py
Line 37: st.markdown("**Zenvra International · Global Transaction Intelligence Platform**")
```

In []:

In []:

Requirements.txt for Cloud

In [5]: **import** os

```
username    = os.getlogin()
DEPLOY_ROOT = rf"C:\Users\{username}\Desktop\finshield_deploy"

requirements = """\
streamlit
pandas
numpy
scikit-learn
xgboost
imbalanced-learn
shap
joblib
matplotlib
"""

req_path = os.path.join(DEPLOY_ROOT, 'requirements.txt')
with open(req_path, 'w') as f:
    f.write(requirements)
print(f" requirements.txt updated →\n    {req_path}")

requirements.txt updated →
C:\Users\DELL\Desktop\finshield_deploy\requirements.txt
```

In []:

Project Summary

Data Generation: A 3,000-row synthetic fintech transaction dataset was created with realistic global coverage and intentional imperfections — missing values, duplicates, inconsistent formatting, and outliers , to mirror real-world data quality challenges.

Data Cleaning: The dataset was systematically cleaned through duplicate removal, median and mode imputation, outlier capping, timestamp parsing, and data type enforcement.

EDA: Seven visualisations uncovered key fraud patterns, elevated fraud in crypto and remittance merchants, higher risk in unverified KYC accounts, night-hour fraud concentration, and regional fraud variation.

Feature Engineering: Log transformations, a composite risk score, ordinal KYC encoding, and one-hot encoding expanded the feature set to 54 model-ready columns.

Modelling: Three classifiers were trained — Logistic Regression, Random Forest, and XGBoost — with SMOTE handling class imbalance. XGBoost won with ROC-AUC of 0.712, Avg Precision of 0.478, and F1 of 0.427.

Deployment: A Streamlit web application was built and deployed publicly via GitHub and Streamlit Community Cloud, accepting 23 transaction inputs and returning instant fraud probability scores with plain-English risk interpretations.

In []: